

Version 2.0

January 2026



OPERATING SYSTEMS PRACTICUM

MODULE 4 - INTRODUCTION TO MANAGEMENT PROCESS

Author:

Ir. Denar Regata Akbi, S.Kom., M.Kom.

Putri Nayla Sabri

Muhammad Zamzam Baihaqi

INFORMATICS LABORATORY

University of Muhammadiyah Malang

TABLE OF CONTENTS

TABLE OF CONTENTS.....	2
INTRODUCTION.....	3
OBJECTIVES.....	3
MODULE TARGETS.....	3
PREPARATION.....	3
KEYWORDS.....	3
THEORETICAL BACKGROUND.....	4
1. FUNDAMENTALS OF PROCESSES.....	4
1.1. Definition and lifecycle of a process.....	4
EXERCISE 1.....	5
1.2. Process states and the Process Control Block (pcb).....	5
1.3. Parent-child relationships, Orphan, and Zombie Processes.....	6
EXERCISE 2.....	8
2. PROCESS MANAGEMENT IN LINUX.....	8
2.1. Identifying processes: PID, 'ps', 'top', AND 'pstree'.....	8
EXERCISE 3.....	10
2.2. Job Control: Foreground, Background, and Suspension.....	10
EXERCISE 4.....	11
2.3. Practical Debugging: using 'systemctl status' and 'journalctl' for service failures.....	11
3. PROCESS CONTROL AND TERMINATION.....	13
3.1. Signals and Process Termination ('kill', 'pkill', 'SIGTERM', 'SIGKILL').....	13
EXERCISE 5.....	15
3.2. Safe Process Handling: avoiding force-kill when possible.....	15
4. CPU SCHEDULING AND PERFORMANCE ANALYSIS.....	17
4.1. Multiprogramming, Multitasking, and the need for Scheduling.....	17
4.2. Core algorithms: FCFS, SJF, Round Robin, Priority Scheduling.....	18
4.3. Performance metrics: gantt charts, waiting time, turnaround time.....	19
PRACTICAL APPLICATIONS.....	21
PRACTICE.....	21
DEMO - CASE STUDY.....	21
ASSESSMENT RUBRIC.....	23
ASSESSMENT SCALE.....	23



INTRODUCTION

OBJECTIVES

1. Understand the lifecycle and states of a process in Linux.
2. Identify, monitor, and manage running processes using CLI tools.
3. Apply safe process control techniques (SIGTERM vs SIGKILL) for system stability.

MODULE TARGETS

1. Identify processes by PID, name, and hierarchy using ps, top, and pstree.
2. Control job execution: run in background, suspend, resume, and terminate processes.
3. Debug service failures using systemctl status and journalctl.
4. Evaluate CPU scheduling concepts and performance metrics (waiting time, turnaround time).

PREPARATION

1. Muslim students may recite the following prayer before starting the lesson
رَضِيتُ بِاللّهِ رَبًّا وَبِالْإِسْلَامِ دِينًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُولًا. رَبِّ زِدْنِي عِلْمًا وَارْزُقْنِي فَهْمًا
2. Laptop or personal computer
3. Rocky Linux
4. Virtual Box
5. Strong motivation and commitment to learning

KEYWORDS

Process, PID, PCB, Parent-Child, Orphan, Zombie, ps, top, pstree, systemctl, journalctl, kill, SIGTERM, SIGKILL, Job Control, CPU Scheduling, FCFS, Round Robin, Waiting Time, Turnaround Time



THEORETICAL BACKGROUND

1. FUNDAMENTALS OF PROCESSES

In a modern **operating system** like **Rocky Linux**, the ability to run multiple programs concurrently is not magic; it is the result of careful process management by the kernel. A **process** is the fundamental unit of work in any operating system. Understanding what a process is, how it behaves, and how it interacts with other processes is essential for both **system administration** and **software development**.

This section introduces the core theoretical concepts that underpin **process behavior in Linux-based systems**, laying the groundwork for practical monitoring, control, and performance analysis in later sections.

1.1. Definition and lifecycle of a process

A. What Is a Process?

A **process** is an instance of a program in execution. While a program is a passive collection of instructions stored on disk (e.g., `/bin/ls`), a process is the active entity created when that program is loaded into memory and executed by the CPU.

A single program (like `chrome`) can spawn multiple processes (e.g., one for the main window, another for a background update). Each process has its own memory space, registers, and execution state.

In Linux, every process is assigned a unique Process ID (PID) by the kernel. This PID is used by the system and administrators to identify, monitor, and control the process throughout its lifetime.

B. The Process Lifecycle

A process goes through a well-defined sequence of stages from creation to termination, known as its **lifecycle**:

1. **New**: The process is being created (e.g., via `fork()` or shell execution).
2. **Ready**: The process is loaded into memory and waiting for the CPU to execute it.
3. **Running**: The process is currently being executed by the CPU.
4. **Waiting (or Blocked)**: The process is paused, waiting for an event (e.g., user input).
5. **Terminated**: The process has finished execution or been forcibly killed.

This lifecycle is managed entirely by the operating system kernel, which ensures fair and efficient use of CPU time among all active processes.



Example:

When you type 'ping google.com' in the terminal:

- A new process is created ('New')
- It enters the ready queue ('Ready')
- The CPU executes it ('Running')
- It waits between pings ('Waiting')
- You press 'Ctrl+C', and it terminates ('Terminated')

EXERCISE 1

Run the command `ping -c 5 google.com`, then observe its process lifecycle stages (New → Running → Waiting → Terminated) using `ps aux | grep ping` in the same time when the ping still running. (Tip: use 2 terminals simultaneously)

1.2. Process states and the Process Control Block (pcb)

A. Process States

At any given moment, a process resides in one of the five states described above. The operating system maintains this state information to coordinate resource allocation and CPU scheduling.

Transitions between states are triggered by events such as:

- **System calls** (e.g., '`read()`', '`write()`')
- **Interrupts** (e.g., timer interrupt, keyboard input)
- **Signals** (e.g., '`SIGTERM`', '`SIGKILL`')

These transitions ensure that no single process monopolizes system resources—a key principle of multitasking operating systems.

B. The Process Control Block (PCB)

The **Process Control Block (PCB)** is a critical data structure maintained by the kernel for every process. It acts as the “identity card” of a process, storing all necessary information to manage and resume its execution.

Key components of the PCB include:

Field	Description
Process State	Current state (e.g., Running, Waiting)
Program Counter (PC)	Address of the next instruction to execute
CPU Registers	Snapshot of register values (for context switching)
CPU Scheduling Info	Priority, scheduling queue pointers
Memory Management Info	Base/limit registers, page tables
Accounting Info	CPU time used, time limits, process number



I/O Status Info	List of open files, allocated devices
Process ID (PID)	Unique numeric identifier
Parent PID (PPID)	PID of the process that created it

When the OS performs a **context switch** (switching from one process to another), it saves the current process's state into its PCB and loads the next process's state from its PCB into the CPU registers.

Why It Matters:

Without the PCB, the OS could not suspend and later resume a process accurately making multitasking impossible.

1.3. Parent-child relationships, Orphan, and Zombie Processes

A. Parent and Child Processes

In Unix-like systems such as Rocky Linux, processes are organized in a **hierarchical tree structure**. When a process creates a new process (typically using the '`fork()`' system call), it becomes the **parent**, and the new process is the **child**.

- Every process (except the first one, '`init`' or '`systemd`') has exactly one parent.
- A parent can have multiple children.
- The relationship is tracked via the **Parent Process ID (PPID)** stored in each child's PCB.

You can view this hierarchy using:

```
praktikumso@localhost:~$ pstree
systemd--ModemManager--3*[{ModemManager}]
          --NetworkManager--3*[{NetworkManager}]
          --SetroubleshootP--3*[{SetroubleshootP}]
          --accounts-daemon--3*[{accounts-daemon}]
```

or

```
praktikumso@localhost:~$ ps -ef
UID      PID    PPID  C  STIME TTY
root         1        0  0  17:58 ?
root         2        0  0  17:58 ?
root         3         2  0  17:58 ?
```



B. Orphan Processes

An **orphan process** is a child whose parent has terminated before it did.

- In most cases, orphaned processes are automatically **adopted by the 'init' process** (PID 1, or 'systemd' in modern Linux).
- This ensures that every process always has a parent, preserving system integrity.

Note:

Orphan ≠ Problem:

Orphaning is normal and safe. For example, background daemons often intentionally orphan themselves to run independently.

C. Zombie Processes

A **zombie process** (or defunct process) is a process that has **completed execution** but still has an entry in the process table because its parent has not yet read its exit status.

- Zombies consume no CPU or memory, but they **do occupy a PID**.
- If too many accumulate, the system may run out of PIDs (though this is rare).

How Zombies Form:

1. Child process finishes and calls `'exit()'`.
2. Kernel keeps minimal info (exit status) in the PCB.
3. Parent must call `'wait()'` or `'waitpid()'` to collect this status.
4. Until then, the child remains a zombie.

Identifying Zombies:

In 'ps' output, zombies appear with a 'Z' in the STAT column:

```
praktikumso@localhost:~$ ps aux | grep 'Z'
```

Note:

Zombie ≠ Harmless Forever:

While a few zombies are normal, persistent zombies indicate a bug in the parent process (e.g., it forgot to call `'wait()'`).

Summary:

Processes are the heartbeat of any operating system. From the moment a command is typed in the terminal to when a service runs in the background, the kernel manages each as a distinct process with a defined lifecycle, state, and identity. Understanding PCBs, parent-child relationships, and abnormal states like zombies prepares you to debug real-world system issues such as unresponsive services or resource leaks using tools like `'ps'`, `'top'`, and `'pstree'`.



EXERCISE 2

Use the command `ps -ef` to find the PID and PPID of your bash process, then compare it with the output of `ps tree -p` to visualize the parent-child relationship.

2. PROCESS MANAGEMENT IN LINUX

In Linux-based systems like Rocky Linux, every running program from a simple `ls` command to a complex web server is represented as a **process**. The operating system kernel assigns each process a unique identifier and manages its execution, memory, and interactions with other processes. Understanding how to **identify**, **monitor**, and **control** these processes is essential for both system administration and troubleshooting.

This section introduces practical tools and techniques used daily by Linux administrators to observe and manage active processes directly from the terminal without relying on graphical interfaces.

2.1. Identifying processes: PID, 'ps', 'top', AND 'pstree'

A. What Is a PID?

Every process in Linux is assigned a **Process ID (PID)**, a unique positive integer that distinguishes it from all other processes. The first process started by the kernel is 'systemd' (or 'init' in older systems), which always has PID 1. All other processes are descendants of this root process.

You can view your current shell's PID using:

```
praktikumso@localhost:~$ echo $$
3234
```

B. The 'ps' Command: Snapshot of Active Processes

The 'ps' (process status) command provides a **static snapshot** of currently running processes. It is the most fundamental tool for process identification.

Common usage:

```
praktikumso@localhost:~$ ps aux
```

- 'a' → show processes for all users
- 'u' → display the process owner and resource usage
- 'x' → include processes not attached to a terminal



Example Output Interpretation:

USER	PID	%CPU	%MEM	VSZ	RSS	TTY
root	1	0.0	1.1	50180	41872	?
root	2	0.0	0.0	0	0	?
root	3	0.0	0.0	0	0	?

- **PID 1234:** A 'vim' process owned by 'praktikumso'
- **STAT "S+":** Sleeping ('S') and in foreground ('+')
- **%CPU / %MEM:** CPU and memory usage

Tip: Use `ps -ef` for a POSIX-compliant format that includes PPID (Parent PID).

C. The 'top' and 'htop' Commands: Real-Time Monitoring

While 'ps' gives a one-time view, 'top' provides a **dynamic, real-time display** of system processes, sorted by CPU usage by default.

To launch:

```
praktikumso@localhost:~$ top
```

Key features:

- Continuously updates every few seconds
- Shows total CPU, memory, and load average
- Allows interactive commands (e.g., 'k' to kill a process)

For a more user-friendly interface, install 'htop':

```
praktikumso@localhost:~$ sudo dnf install htop -y
```

```
praktikumso@localhost:~$ htop
```

'htop' adds:

- Color-coded output
- Vertical and horizontal scrolling
- Mouse support (if enabled)
- Tree view of parent-child relationships

Best Practice: Use 'top' for quick checks; use 'htop' for deeper analysis during labs.

D. The 'pstree' Command: Visualizing Process Hierarchy

Processes in Linux form a **tree structure**, where each child process inherits from a parent. The 'pstree' command displays this hierarchy visually.



Example:

```
praktikumso@localhost:~$ pstree -p
```

Output snippet:

```
systemd(1)─sshd(890)─sshd(1200)─bash(1201)─vim(1234)
      │
      └─httpd(900)─httpd(901)
                └─httpd(902)
```

- PID shown in parentheses
- Clearly shows that '**vim(1234)**' is a child of '**bash(1201)**', which is under '**sshd**'

Note: This is invaluable for identifying orphaned or misbehaving process chains.

EXERCISE 3

Run **top**, press **1** to view per-core CPU usage, then identify the process with the highest load and record its **%CPU**, **PID**, and state (**S**, **R**, etc.).

2.2. Job Control: Foreground, Background, and Suspension

Linux shells (like Bash) support **job control**, allowing users to manage multiple processes within a single terminal session.

A. Running Jobs in the Background

Append **&** to a command to run it in the background:

```
praktikumso@localhost:~$ sleep 60 &
```

Output:

```
[2] 8217
```

- '**[2]**' = job number
- '**8217**' = PID of the background process

You can continue using the terminal to execute other commands while the job runs.

B. Listing and Managing Jobs

Use the **'jobs'** command to list all jobs in the current shell:

```
praktikumso@localhost:~$ jobs
[1]+  Stopped                  vim test.sh
[2]-  Done                     sleep 60
```

Moving Jobs Between Foreground and Background

- Bring a job to the foreground:

```
praktikumso@localhost:~$ fg %1
vim test.sh
```



- Suspend a running foreground job: Press 'Ctrl + Z' This sends the 'SIGTSTP' signal and pauses the process.
- Resume a suspended job in the background:

```
praktikumso@localhost:~$ bg %1
```

Note: Job control only works for processes started from the **current shell session**. It does not affect system services like 'sshd' or 'httpd'.

C. Practical Example: Managing a Long-Running Script

1. Create a script:

```
praktikumso@localhost:~$ echo 'for i in {1..10}; do echo "step $i"; sleep 2; done' > longtask.sh
praktikumso@localhost:~$ chmod +x longtask.sh
```

2. Run it in the background:

```
praktikumso@localhost:~$ ./longtask.sh &
[1] 8470
```

3. Check jobs:

```
praktikumso@localhost:~$ jobs
```

4. Suspend and resume:

```
praktikumso@10:~$ fg %1
```

Press Ctrl+Z

```
praktikumso@10:~$ bg %1
```

5. Terminate if needed:

```
praktikumso@10:~$ kill %1
```

EXERCISE 4

Run `sleep 30 &`, then use `jobs`, `fg`, and `Ctrl+Z` to move the process between foreground, background, and suspended states.

2.3. Practical Debugging: using 'systemctl status' and 'journalctl' for service failures

In modern Linux distributions like Rocky Linux, most system services (e.g., SSH, Apache, MySQL) are managed by **systemd**. When a service fails to start, two key commands help diagnose the issue.



A. Step 1: Check Service Status with 'systemctl status'

Example: Debugging a failed SSH service after changing its port.

```
praktikumso@10:~$ sudo systemctl status sshd
```

Typical output includes:

- Whether the service is **active (running)** or **inactive (dead)**
- The **main PID** (if running)
- **Recent log lines** showing the last few events
- Exit code and error message (e.g., "Address already in use")

Note: If the service failed, look for red-colored lines or messages like:
Failed to start **OpenSSH** server daemon.

B. Step 2: Inspect Detailed Logs with 'journalctl'

The '**journalctl**' command accesses the **systemd journal**, which stores structured logs for all services.

- To view logs for 'sshd':

```
praktikumso@10:~$ sudo journalctl -u sshd
```

- For real-time monitoring:

```
praktikumso@10:~$ sudo journalctl -u sshd -f
```

- To see only the last 20 lines:

```
praktikumso@10:~$ sudo journalctl -u sshd --since "5 minutes ago"
```

Common Error Patterns:

Error Message	Likely Cause
Address already in use	Port conflict (e.g., another app uses port 2222)
Permission denied	SELinux blocking the port
Configuration file error	Syntax mistake in /etc/ssh/sshd_config

C. Step 3: Fix and Restart

After identifying the issue:

1. Edit the config file (e.g., '**/etc/ssh/sshd_config**')
 2. Test syntax:

```
praktikumso@10:~$ sudo ssh -t
```

3. If valid, restart:

```
praktikumso@10:~$ sudo systemctl restart sshd
```



Always verify:

```
praktikumso@10:~$ sudo systemctl restart sshd
```

D. Why This Matters for Students

- Real-world servers rarely have GUIs—debugging happens in the terminal.
- Understanding '`systemctl`' and '`journalctl`' is essential for DevOps, cloud roles, and system administration.
- These tools are standard across RHEL, CentOS, Fedora, and Rocky Linux.

Tip: Combine with '`grep`' for faster filtering:

```
journalctl -u sshd | grep "Failed"
```

Summary:

Linux provides powerful, CLI-based tools to **observe**, **control**, and **debug** processes. From simple commands like '`ps`' to advanced log inspection with '`journalctl`', mastering these utilities prepares you to manage real systems confidently exactly as expected in professional environments.

3. PROCESS CONTROL AND TERMINATION

In Linux, processes do not run indefinitely; they must eventually end, either by completing their task or being stopped by the system or user. The operating system provides precise mechanisms to control, interrupt, and terminate processes safely. These mechanisms are based on signals, a form of inter-process communication that allows the kernel or other processes to notify a process of an event.

Understanding how to send signals and when to use gentle termination versus forceful killing is essential for responsible system administration and debugging.

3.1. Signals and Process Termination ('kill', 'pkill', 'SIGTERM', 'SIGKILL')

A. What Is a Signal?

A **signal** is a software interrupt delivered to a process to notify it of an important event. Signals can originate from:

- The kernel (e.g., segmentation fault),
- Other processes (e.g., a parent terminating a child),
- The user (e.g., pressing '`Ctrl+C`').



Each signal has a **name** and a **number**. Common signals include:

Signal	Number	Default Action	Description
<code>SIGTERM</code>	15	Terminate gracefully	Request to shut down cleanly
<code>SIGKILL</code>	9	Force kill (cannot be caught or ignored)	Immediate termination
<code>SIGINT</code>	2	Terminate	Sent by 'Ctrl+C'
<code>SIGSTOP</code>	19	Pause	Suspend process (cannot be ignored)
<code>SIGCONT</code>	18	Resume	Continue a paused process

Core understanding:

Only '`SIGKILL`' and '`SIGSTOP`' **cannot be handled, blocked, or ignored** by a process. All others can be caught and responded to allowing programs to clean up before exiting.

B. The '`kill`' Command

Despite its name, '`kill`' does **not always terminate** a process, it sends a **signal**.

Basic syntax:

```
praktikumso@10:~$ kill [signal] PID
```

Examples:

Send `SIGTERM` (default)

```
praktikumso@10:~$ kill 1234
```

Send `SIGKILL` (force kill)

```
praktikumso@10:~$ kill -9 1234
```

Send `SIGTERM` explicitly

```
praktikumso@10:~$ kill -15 1234
```

Best Practice: Always try '`SIGTERM`' first. Only use '`SIGKILL`' if the process does not respond.

The '`pkill`' and '`killall`' Commands

When you don't know the PID, you can target processes by name:

- '`pkill`': Sends a signal to processes matching a pattern.

```
praktikumso@10:~$ pkill firefox
```

```
praktikumso@10:~$ pkill -f "python script.py"
```

- '`killall`': Terminates all processes with a given name.



```
praktikumso@10:~$ killall httpd
```

Caution: 'killall' on some systems (like Solaris) kills **all processes**—but in Linux, it only affects processes by name. Still, use with care.

C. Practical Example: Graceful vs. Forceful Termination

1. Start a long-running process:

```
praktikumso@10:~$ sleep 300 &
[1] 10293
```

Note the PID (e.g., '10293').

2. Try graceful termination:

```
praktikumso@10:~$ kill 10293
[1]+  Terminated                  sleep 300
```

→ The 'sleep' process exits immediately (it handles 'SIGTERM').

3. Now start another:

```
praktikumso@10:~$ sleep 300 &
[1] 10334
```

4. If it were unresponsive, force kill:

```
praktikumso@10:~$ kill -9 10334
[1]+  Killed                        sleep 300
```

Use 'ps' afterward to confirm the process is gone.

EXERCISE 5

Run `sleep 60 &`, note its PID, send a `SIGTERM` signal using `kill`, and verify if the process terminates. If it persists, forcefully terminate it using `SIGKILL`.

3.2. Safe Process Handling: avoiding force-kill when possible

A. Why Avoid 'SIGKILL'?

Using '`kill -9`' may seem effective, but it comes with risks:

- **No cleanup:** Open files, network connections, or temporary data may not be closed properly.
- **Data corruption:** Databases or log files might be left in inconsistent states.
- **Poor user experience:** Applications cannot save session state or notify users.

Rule of Thumb:

Never start with 'SIGKILL'. Always attempt a graceful shutdown first.



B. Recommended Termination Workflow

1. Identify the process:

```
ps aux | grep nginx
```

OR

```
pgrep nginx
```

1. Send 'SIGTERM' (graceful request):

```
kill $(pgrep nginx)
```

2. Wait 5–10 seconds. Check if it's gone:

```
pgrep nginx
```

3. Only if still running, send 'SIGKILL':

```
pkill -9 nginx
```

This approach is used by system services like `'systemctl stop nginx'`.

C. Real-World Case: Stopping a Web Server

Suppose Apache ('`httpd`') is misbehaving.

Correct way:

```
praktikumso@10:~$ sudo systemctl stop httpd
```

→ This sends 'SIGTERM', waits, then 'SIGKILL' if needed.

Incorrect way:

```
praktikumso@10:~$ sudo kill -9 $(pgrep httpd)
```

→ May leave sockets open or logs incomplete.

D. Signal Handling in Scripts

You can also **trap signals** in Bash scripts to perform cleanup:

```
1 #!/bin/bash
2
3 cleanup(){
4     echo "Shutting down gracefully..."
5     rm -f /tmp/myapp.lock
6     exit 0
7 }
8
9 #Trap SIGTERM and SIGINT
10 trap cleanup TERM INT
11
12 # Simulate work
13 while true: do
14     sleep 1
15 done
```

Now, '`kill [PID]`' or '`Ctrl+C`' will trigger the 'cleanup' function.

This is a professional practice used in production daemons.



Summary:

Process termination is not just about stopping a program it's about **doing so responsibly**. By understanding signals and using `'SIGTERM'` before `'SIGKILL'`, you ensure system stability, data integrity, and professional-grade administration. These skills are directly applicable when managing services like `'sshd'`, `'httpd'`, or custom scripts in Rocky Linux.

4. CPU SCHEDULING AND PERFORMANCE ANALYSIS

In a modern operating system like Rocky Linux, multiple processes often compete for limited CPU resources. Without a systematic way to decide which process runs next, the system would become inefficient, unresponsive, or even unfair. This is where CPU scheduling comes into play.

CPU scheduling is the mechanism by which the operating system kernel selects the next process to execute on the CPU. It ensures that system resources are shared fairly, response time is minimized, and throughput is maximized—especially in multi-user or server environments.

This section introduces the foundational concepts of scheduling, explores core algorithms used in practice and education, and provides tools to quantitatively evaluate their performance using standard metrics.

4.1. Multiprogramming, Multitasking, and the need for Scheduling

A. What Is Multiprogramming?

Multiprogramming is the technique of keeping multiple processes in memory at the same time. When one process is waiting (e.g., for I/O), the CPU can switch to another ready process—maximizing CPU utilization.

Example: While a backup script waits for disk writes, the system can still respond to SSH login requests.

B. What Is Multitasking?

Multitasking extends multiprogramming by rapidly switching between processes, giving users the illusion that many programs are running **simultaneously**, even on a single-core CPU.

- In Linux, this is achieved through **time-sharing**: each process gets a small slice of CPU time.
- The OS kernel uses a scheduler to manage this switching.

C. Why Is Scheduling Necessary?

Without scheduling:

- A single CPU-bound process could monopolize the CPU indefinitely.
- Interactive applications (like terminals or web servers) would become unresponsive.



- System fairness and efficiency would collapse.

Thus, the scheduler's role is critical: it balances **throughput**, **latency**, **fairness**, and **resource utilization**.

4.2. Core algorithms: FCFS, SJF, Round Robin, Priority Scheduling

Several scheduling algorithms exist, each with trade-offs. Below are four fundamental algorithms covered in the RPS and commonly used in academic and real-world contexts.

A. First-Come, First-Served (FCFS)

- **Principle:** Processes are executed in the order they arrive.
- **Type:** Non-preemptive
- **Pros:** Simple to implement.
- **Cons:** Can lead to **convoy effect**, short processes wait behind long ones.

Example:

- Processes arrive: P1 (10 ms), P2 (2 ms), P3 (5 ms)
- Execution order: P1 → P2 → P3
- P2 and P3 wait unnecessarily.

B. Shortest Job First (SJF)

- **Principle:** The process with the shortest **burst time** (CPU time needed) runs first.
- **Type:** Can be **non-preemptive** (SJF) or **preemptive** (Shortest Remaining Time First, SRTF).
- **Pros:** Minimizes average waiting time (optimal if burst times are known).
- **Cons:** Requires prior knowledge of burst time; may cause **starvation** of long processes.

For the real-world analogy:

A cashier serving customers with fewer items first.

C. Round Robin (RR)

- **Principle:** Each process gets a fixed time slice called a **time quantum** (e.g., 2 ms or 5 ms).
- **Type:** Preemptive
- **Pros:** Fair; no starvation; good for interactive systems.
- **Cons:** High context-switching overhead if quantum is too small; poor CPU utilization if too large.

In Linux:

The default scheduler (CFS – Completely Fair Scheduler) uses a dynamic, fair-share variant of RR.



D. Priority Scheduling

1. **Principle:** Each process is assigned a priority; the highest-priority process runs first.
2. **Type:** Can be preemptive or non-preemptive.
3. **Pros:** Useful for real-time or critical tasks.
4. **Cons:** Low-priority processes may **starve** indefinitely.

Solution: **Aging gradually** increases the priority of waiting processes to prevent starvation.

Priority values:

- Lower number = higher priority (e.g., 0 = highest, 10 = lowest)
- Or vice versa depends on system design.

4.3. Performance metrics: gantt charts, waiting time, turnaround time

To compare scheduling algorithms objectively, we use quantitative metrics.

A. Gantt Charts

A **Gantt chart** visually represents the execution timeline of processes over time.

Example (FCFS):

```
|----P1----|---P2---|-----P3-----|
0         10      12         17
```

Gantt charts help visualize:

- CPU idle time
- Process waiting periods
- Preemption points (in RR or SRTF)

B. Waiting Time (WT)

Waiting Time = Total time a process spends in the **ready queue** before getting the CPU.

Formula:

$$WT = \text{Turnaround Time} - \text{Burst Time}$$

Goal: Minimize average waiting time across all processes.

C. Turnaround Time (TAT)

Turnaround Time = Total time from **process arrival** to **completion**.

Formula:

$$TAT = \text{Completion Time} - \text{Arrival Time}$$

Goal: Minimize average turnaround time for better system responsiveness.



D. Calculating Average Metrics

Given a set of processes:

Process	Burst Time	Arrive Time
P1	10	0
P2	2	3
P3	5	4

For **FCFS**:

- P1: $TAT = 10 - 0 = 10$, $WT = 10 - 10 = 0$
- P2: $TAT = 12 - 3 = 9$, $WT = 9 - 2 = 7$
- P3: $TAT = 17 - 4 = 13$, $WT = 13 - 5 = 8$

→ **Average WT** = $(0 + 7 + 8) / 3 = 5.0$

→ **Average TAT** = $(10 + 9 + 13) / 3 = 10.67$

E. Practical Connection to Rocky Linux

While you won't manually apply FCFS or SJF in Rocky Linux (the kernel uses **CFS**), understanding these algorithms helps you:

- Interpret '**top**'/'**htop**' output,
- Tune process priorities with '**nice**' and '**renice**',
- Diagnose performance bottlenecks in scripts or services.

Command to view process priority:

```
praktikumso@10:~$ ps -eo pid,comm,pri,ni
```

- '**ni**' = nice value (lower = higher priority)

Summary:

CPU scheduling is not just theory—it's the engine behind every responsive Linux system. By mastering algorithms like FCFS, SJF, RR, and Priority, and learning to compute metrics like waiting time and turnaround time, you gain the analytical tools to evaluate system performance and make informed decisions—exactly as required by the RPS and real-world system administration.



PRACTICAL APPLICATIONS

PRACTICE

Objective: Students directly try basic commands to observe, manage, and debug processes in Rocky Linux.

(The exercise is not shown yet; students wait for instructions during the practice session.)

DEMO - CASE STUDY

The Situation

You are a system administrator for a campus lab. A student reports two issues on the Rocky Linux server:

1. An unintended background process is consuming high CPU.
2. The **SSH** service fails to restart after a configuration change—and they don't know how to diagnose the error.

Your task: fully restore the system using only CLI commands—without any module notes, GUI tools, or preloaded files.

DEMO TASKS (Student Must Perform Live in Terminal)

1. Create a High-CPU Background Process

- Simulate a stuck process by running:

```
praktikumso@localhost:~$ yes > /dev/null &
```

- Note the **job number** and **PID** shown (e.g., `[1] 1234`).

2. Identify and Observe the Process

- Use `ps aux` or `top` to locate the `yes` process.
- Use `pstree -p` to confirm its parent-child relationship.
- Observe its CPU usage in `top` or `htop`.

3. Terminate the Process Safely

- Send `SIGTERM` using `kill [PID]`.
- If it's still running after 5 seconds, send `SIGKILL` (`kill -9 [PID]`).
- Verify the process is gone with:

```
praktikumso@localhost:~$ ps aux | grep yes
```

4. Simulate an SSH Service Failure

- Add an invalid line to the SSH config:

```
praktikumso@localhost:~$ echo "port abc" | sudo tee -a /etc/ssh/sshd_config
```



- Restart SSHD
- Confirm it failed Status of sshd:

5. Diagnose and Fix the Issue

- Check logs of `sshd` (using `journalctl`) since 1 minute ago
- Identify the invalid line (`Port abc`).
- Remove it
`sudo sed -i '/Port abc/d' /etc/ssh/sshd_config`
- Validate syntax:
`sudo sshd -t`
- Restart SSH
- Verify SSH active(clue: `systemctl`)

6. Demonstrate Job Control (Optional)

- Run:

```
praktikumso@localhost:~$ sleep 60 &
```

- List jobs: ``jobs``
- Suspend with ``Ctrl+Z``, then resume in background: ``bg``
- Terminate with: ``kill %1``

Demo Rules

1. No Module or Notes Access

Students must not open the module PDF, personal notes, or online documentation.

2. All Tasks Done Live in Terminal

No pre-loaded files or scripts—everything is created and fixed during the demo.

3. Direct Observation by Instructor/Assistant

The assessor will evaluate:

- Logical workflow (identify → control → verify),
- Proper use of ``SIGTERM`` before ``SIGKILL``,
- Ability to interpret ``journalctl`` output and fix configs.

4. Mandatory Verification

Every step must be confirmed using commands like:

- ``ps``, ``systemctl is-active``, ``sshd -t``

5. Follow Live Instructions

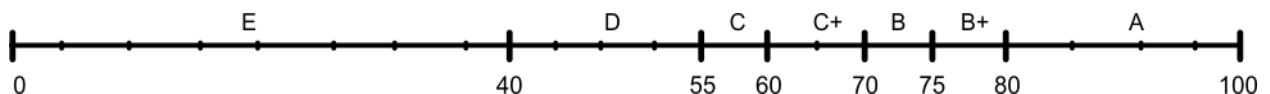
Students must respond interactively to the instructor's directions.



ASSESSMENT RUBRIC

Assessment Aspect	Points (Total 100%)
EXERCISE	Total 5%
Exercise 1	1%
Exercise 2	1%
Exercise 3	1%
Exercise 4	1%
Exercise 5	1%
PRACTICE	Total 20%
Practice 1	10%
Practice 2	10%
DEMO	Total 75%
Case Study Completion	35%
Understanding Process Monitoring & Analysis	10%
Understanding Signal Handling Mastery	10%
Understanding Service & Log Diagnostics	10%
Understanding Job Control Proficiency	10%

ASSESSMENT SCALE



- A** = (81 - 100) → **Sepuh**
- B+** = (75 - 80) → **Very Good**
- B** = (70 - 74) → **Good**
- C+** = (60 - 69) → **Fairly Good**
- C** = (55 - 59) → **Fair**
- D** = (41 - 54) → **Poor**
- E** = (0 - 40) → **Bro really...**

